

Super Admin Console

Integration & Implementation Guide

From Static Prototype to Dynamic Multi-Tenant ERP — Intern-Level Walkthrough

Stack	Next.js 14 · TypeScript · Django REST Framework · PostgreSQL (django-tenants)
Audience	Junior / Intern engineers joining the Eskoolia ERP project
Screens covered	5 screens — Dashboard · School Management · Billing · Audit Log · Global Policies
Version	v1.0 · May 2026
Classification	Confidential — Internal Engineering Document

#	Section	Topic
01	Project Context	What Eskoolia is, multi-tenancy model, tech stack, and design rules
02	Screen Explanations	All 5 Super Admin screens — purpose, components, and current hardcoded data
03	Data Residency Concepts	Tenant provisioning, DB isolation, S3 storage, DPDP compliance (from PDF brief)
04	Integration Architecture	How the console fits into the ERP — authentication, API design, subdomain routing
05	Static → Dynamic Conversion	Exactly what to replace in each screen and which API endpoint to call
06	Advantages of the Screens	Why this design is production-ready and what it saves in development time
07	Impact on Existing Code	Which files to touch, which to leave alone, and what to add
08	V1 vs V2 Screens	What to ship first and what to defer — with justification
09	Implementation Route-Map	Sprint-by-sprint plan — step-by-step guide for an intern to implement
10	Audit & Validation Checklist	How to verify each screen is production-ready before going live

01

Project Context

What Eskoolia is, the architecture, design contract, and coding rules every engineer must know

What Is Eskoolia?

Eskoolia is a multi-tenant, cloud-hosted School ERP that replaces paper registers and Excel sheets in schools across India. A single codebase serves 700–800+ students per school (potentially 1000+), from Nursery to Grade 10 or 12, across multiple campuses. The goal is best-in-market quality — Google/Notion-level UI, zero manual data entry, and fully automated workflows. Schools range from single-campus CBSE schools to multi-campus enterprise groups under one subscription.

Multi-Tenancy Architecture

Each school is a separate **tenant** — same codebase, but each school's data lives in its own PostgreSQL schema (via **django-tenants**). The backend automatically routes every query to the correct schema based on the subdomain in the request. This means:

Rule	What It Means for You as a Developer
Never hardcode school name, class lists, sections, fees	Always fetch from /api/school/config/, /api/classes/, /api/academic-years/
Backend scopes all queries automatically	Do NOT add school_id filters in frontend — the DB router handles it
Academic year is always a parameter	Admissions/students/fees for 2025-26 are completely separate from 2026-27
Tenant ID is immutable after provisioning	Once generated (TNT_XXXXXXX), it can never be changed — it is the PK for all isolation
Every new school gets its own PostgreSQL schema	DROP SCHEMA school_x CASCADE completely deletes a school — clean and auditable

Tech Stack

Layer	Technology	Notes
Frontend	Next.js 14 App Router · TypeScript (strict)	No any types. Props must be typed.
Styling	Tailwind CSS · tokens.css	Never use globals.css old variables. Use --pu, --ink-1, --bd etc.
Animations	Framer Motion	Shimmer skeletons on all async loads — never bare spinners
Charts	Recharts	Used in dashboard KPI sparklines and analytics
Backend	Django REST Framework · Python	DRF serializers for every API response shape
DB	PostgreSQL with django-tenants	Schema-per-tenant. SQLite for local dev only.
Background jobs	Celery + Redis	For auto-reminders, bulk sends, scheduled reports
Messaging	WhatsApp Cloud API / Gupshup	Parent notifications, admission follow-ups
File storage	AWS S3 (ap-south-1)	Tenant-scoped paths: tenants/{tenant_id}/...

Universal Page Design Contract

Every single page in Eskoolia follows this exact shell. Only the content changes. Never break this structure — users build muscle memory from consistent patterns.

Zone	What Goes Here	Tailwind / Token
Page header row	Bold+Italic-accent title · subtitle in gray · action buttons right-aligned	font-bold + text-indigo-500 font-light
Stats row	4 metric cards with KPI value, trend, footer label	rounded-xl border border-gray-100 bg-white p-4
White outer card	All content sections live inside this single card	rounded-2xl shadow-sm border border-gray-100 bg-white p-6
Section 01/02/03	Collapsible with chevron. State stored in localStorage	Numbered label + collapse arrow
Active filter pills	Selected state	bg-indigo-600 text-white rounded-full px-3 py-1 text-sm
Inactive pills	Unselected state	bg-gray-100 text-gray-600 rounded-full px-3 py-1 text-sm
Primary accent	Buttons, tabs, active states	--pu: #6D4AFF (tokens.css)
Error color	Validation, overdue, danger	--err: #E0463A

02 Screen-by-Screen Explanation

All 5 screens of the Super Admin Console — what each does, its components, and what is currently hardcoded

The Super Admin Console is the platform-level control panel that sits **above** all school tenants. School principals and staff never see it. Only Eskoolia's own platform admins (like "Adwait Rao" in the prototype) can access it. It has 5 screens reachable via hash routing (`#/dashboard`, `#/schools`, `#/billing`, `#/audit`, `#/policies`).

Screen 1 — Super Admin Dashboard		#/dashboard	V1 — ESSENTIAL
PURPOSE	Cross-tenant overview of every school on the platform. Gives a single glance at total schools, total students, monthly recurring revenue (MRR), and alerts that need attention. Contains a "Schools by board" bar chart (CBSE / SSC AP / ICSE / SSC TG mix) and a "Recent activity" list showing the last events across all tenants.		
KEY COMPONENTS	4 KPI cards (Total Schools, Students Served, Monthly Recurring, Needs Attention) · Schools-by-board bar chart · Recent activity feed · Export button · Add School shortcut		
CURRENTLY HARDCODED	SCHOOLS array (9 hardcoded school objects with students, staff, mrr, status) · KPI trend text ("+842 MoM", "+12.4%") · Board breakdown percentages · Activity timestamps		

Screen 2 — School Management		#/schools	V1 — ESSENTIAL
PURPOSE	Provision, monitor, and manage every school tenant. Each school card shows its Tenant ID, GSTIN, UDISE code, PAN, board affiliation, plan tier, seat usage bar, and last activity. Contains smart filter chips (status / board / plan / region / state) and a search input for name, tenant ID, or GSTIN. Clicking a school card opens a detail panel with full provisioning fields (DB shard, storage region, SSO method, API access toggle).		
KEY COMPONENTS	KPI row (Total, Active, On Trial, Needs Attention) · Smart filter bar · School cards grid · Tenant detail panel · "Add School" provisioning form with all django-tenants fields		
CURRENTLY HARDCODED	SCHOOLS array (all 9 school objects) · Filter state (no real filtering logic) · Seat usage bars (static .71, .62, .55...) · Activity timestamps · Plan badges		

Screen 3 — Tenants & Billing		#/billing	V1 — ESSENTIAL
PURPOSE	GST-compliant invoicing across all schools. The seller is Eskoolia HQ (Karnataka, GSTIN 29AABCE1234F1ZS). IGST applies for inter-state buyers; CGST+SGST for intra-state. Shows an invoice list with status badges (Paid/Sent/Overdue), features one full sample invoice (line items with SAC codes, GST calculation, amount in words), and provides GSTR-1 export. Also shows MRR, GST collected, outstanding, and at-risk metrics.		
KEY COMPONENTS	4 KPI cards (MRR, GST Collected, Outstanding, At-Risk) · Invoice list table · Full GST invoice preview (SAC 998313, IGST/CGST/SGST split, amount in words) · Export GSTR-1 button · New Invoice button		
CURRENTLY HARDCODED	Invoice list (6 hardcoded invoice objects) · Seller GSTIN / state hardcoded · MRR trend text · GST amounts · Line items and SAC codes · "Amount in words" calculation		

Screen 4 — Audit Log		#/audit	V1 — ESSENTIAL
PURPOSE	Append-only record of every super-admin and system action across all tenants. Required for data isolation compliance. Logs are retained for 7 years. Events include school.provision, plan.upgrade, auth.impersonate, invoice.overdue, api_key.rotate, school.archive, and backup.complete. Each entry shows timestamp, actor, action code, detail description, and source IP address.		
KEY COMPONENTS	Filter panel (actor / action type / tenant / date range) · Audit event list with color-coded icons (ok=green, warn=amber, danger=red) · Export CSV button · Refresh button		
CURRENTLY HARDCODED	10 hardcoded audit event objects · All timestamps are static May 2026 dates · IP addresses hardcoded · Filter chips have no real filtering logic		

Screen 5 — Global Policies		#/policies	V2 — DEFER
PURPOSE	Platform-wide defaults inherited by every tenant. Individual schools can override only where the policy is marked "overridable". Policies are grouped into three categories: Security & Authentication (password rules, MFA, session timeout), Data Isolation & Retention (RLS policy, backup days, archive grace, audit retention), and Billing & Compliance (GST seller state, rate, invoice prefix, due days, dunning).		
KEY COMPONENTS	3 policy groups as collapsible panels · Each policy shows a monospace key (e.g. password.min_length), a description, current value, and a toggle switch · Save Changes button · Export Config button		
CURRENTLY HARDCODED	All 14 policy entries with their values (hardcoded objects) · Toggle state (all default "on") · No persistence — changes are lost on refresh		

03

Data Residency & Tenant Provisioning Concepts

Summary of the Engineering Brief PDF — the theory behind the implementation

What Is Tenant Provisioning?

When a new school signs up to Eskoolia, the system must create a completely isolated environment for that school in one atomic operation. This is called **tenant provisioning**. It allocates a unique Tenant ID, creates a new PostgreSQL schema, assigns an S3 storage path, configures the subdomain, and sets up authentication. The recommended approach is **schema-per-tenant using django-tenants** — one PostgreSQL server hosts all schools, but each school's data is invisible to others at the DB level.

Strategy	Isolation	Cost	Recommended for Eskoolia?
Shared Schema (tenant_id FK)	Low — risk of data leaks	Cheapest	No — unsafe
Schema-per-Tenant (django-tenants)	High — separate PG schema	Medium	Yes — use this
DB-per-Tenant (separate databases)	Maximum	Very expensive	Future: enterprise only

The SchoolTenant Model

This is the Django model that backs the School Management screen. Every field in the provisioning form maps directly to a field here:

Field	Type	What It Maps To in the UI
tenant_id	CharField (auto, immutable)	Read-only TNT_ display in school card header
name	CharField	"School Name" input in provisioning form
subdomain_url	CharField (unique)	"Subdomain" input — locked after provisioning
shard_region	CharField	"DB Shard" dropdown (shard-ap-south-1a etc.)
storage_region	CharField	"Storage Region" dropdown (S3 ap-south-1 etc.)
backup_retention	IntegerField (default 30)	"Backup Retention" number input (days)
sso_method	CharField (choices)	"SSO / Auth" dropdown (native/google/microsoft/saml)
api_access	BooleanField (default False)	"API Access" toggle — off by default
plan	CharField (choices)	"Plan" badge (trial/premium/enterprise)
status	CharField (default onboarding)	Status chip on school card
provisioned_at	DateTimeField (auto)	"Added on" timestamp in audit trail

Key Concepts for Interns

Row-Level Security (RLS)

django-tenants switches the PostgreSQL search_path on each request. Student.objects.all() automatically returns ONLY the current school's students. You never need to add a WHERE tenant_id=? filter. The DB router handles it invisibly.

Subdomain Routing

greenwood.eskoolia.com → middleware reads "greenwood", looks up the matching SchoolTenant, sets the schema, and your view runs against Greenwood's data. The superadmin console uses app.eskoolia.com (the public schema) and sees all tenants.

Deleting a Tenant

`DROP SCHEMA school_greenwood CASCADE` — one SQL command removes all of a school's data permanently. This is DPDP Act 2023 compliant (Right to Erasure).

IGST vs CGST+SGST

Eskoolia HQ is in Karnataka (state 29). If a school is in any other state, it's inter-state → IGST at 18%. If the school is also in Karnataka → CGST 9% + SGST 9%. The billing screen auto-detects this from the buyer's state code.

04

Integration Architecture

How the Super Admin Console plugs into the existing Eskoolia ERP without breaking anything

Where the Console Lives

The Super Admin Console is a **separate Next.js route group** under (super-admin). It sits completely outside the regular school dashboard (which is under (dashboard)). This isolation means:

Concern	How It Is Handled
Authentication	Separate JWT with role="super_admin". Middleware checks this role before any super-admin route renders. School-level JWTs are rejected.
Data access	Super-admin API endpoints query the PUBLIC schema (which holds all SchoolTenant records). They never switch into a school schema.
URL isolation	Super-admin routes live at /super-admin/* and are protected by a separate middleware group. School routes live at /dashboard/*.
Existing code impact	Zero breaking changes. You are adding new files, not editing existing ones. The school dashboard continues to work exactly as before.
Shared components	You can reuse Button, Badge, Input, Table, and KPICard components from the existing component library — no new design system needed.

API Endpoint Map

Screen	Method	Endpoint	Returns
Dashboard	GET	/api/super-admin/dashboard/	schools[], totalStudents, totalStaff, mrr, alerts
School Management	GET	/api/super-admin/schools/	Paginated SchoolTenant list with filters
School Management	POST	/api/super-admin/schools/provision/	tenant_id, status: provisioned
School Management	PATCH	/api/super-admin/schools/{tenant_id}/	Updated SchoolTenant
School Management	DELETE	/api/super-admin/schools/{tenant_id}/	status: archived/deleted
Billing	GET	/api/super-admin/billing/invoices/	Paginated invoice list with GST fields
Billing	POST	/api/super-admin/billing/invoices/	New GST invoice with computed tax
Billing	GET	/api/super-admin/billing/mrr/	MRR, GST collected, trend data
Audit Log	GET	/api/super-admin/audit/	Paginated audit events with filters
Global Policies	GET	/api/super-admin/policies/	All policy key-value pairs with metadata
Global Policies	PATCH	/api/super-admin/policies/	Updated policy values, audit logged

Next.js Folder Structure

```

app/
  (dashboard)/          ← existing school ERP (untouched)
  (super-admin)/        ← NEW: add this route group
  layout.tsx            ← checks role=super_admin, redirects if not
  super-admin/
    page.tsx             ← redirects to /super-admin/dashboard
    dashboard/page.tsx  ← Screen 1
    schools/
      page.tsx           ← Screen 2 (list)
      [tenantId]/page.tsx ← School detail
      billing/page.tsx   ← Screen 3
      audit/page.tsx     ← Screen 4
      policies/page.tsx  ← Screen 5 (V2)
lib/
  api/
    super-admin/        ← NEW: API client functions
      schools.ts
      billing.ts
      audit.ts
      policies.ts

```

Authentication Flow

The super-admin login is a separate endpoint (`/api/auth/super-admin/token/`) that returns a JWT with role: "super_admin" in the payload. The Next.js `layout.tsx` for the (super-admin) route group reads this token and redirects to the main login if the role does not match. All API calls from the super-admin console must pass this token in the `Authorization: Bearer <token>` header. On the Django side, use a custom `IsSuperAdmin` permission class (not `IsAuthenticated`) on every super-admin view.

05 Static → Dynamic Conversion

Exactly what to replace in each screen, which API to call, and the TypeScript interface to define

The prototype HTML uses a hardcoded SCHOOLS array with 9 school objects. Every number, name, and status in the UI is derived from this static data. Your job is to replace this array with a real API call — and then the rest of the screen works exactly the same way, because the rendering logic is already written.

Screen 1 — Dashboard: What to Replace

Hardcoded Value	Replace With	API Call
SCHOOLS.length	schools.total from API	GET /api/super-admin/dashboard/
SCHOOLS.reduce(...students)	dashboard.totalStudents	same call above
SCHOOLS.reduce(...mrr)	dashboard.mrr.current	same call above
SCHOOLS.filter(active).length	dashboard.activeSchools	same call above
"2" (Needs Attention)	dashboard.alertCount	same call above
CBSE/SSC bar percentages	dashboard.boardBreakdown[]	same call above
Recent activity feed items	recentEvents[] from audit log	GET /api/super-admin/audit/?limit=5
"+842 MoM", "+12.4%" trends	dashboard.trends.students, dashboard.trends.mrr	same dashboard call

TypeScript Interface — Dashboard

```
// lib/api/super-admin/dashboard.ts
export interface DashboardData {
  totalSchools: number;
  activeSchools: number;
  totalStudents: number;
  totalStaff: number;
  mrr: { current: number; trend: number }; // trend as %
  alertCount: number;
  boardBreakdown: { board: string; count: number; percent: number }[];
  trends: { students: number; mrr: number };
}

export async function getDashboard(): Promise<DashboardData> {
  const res = await fetch(
    `${process.env.NEXT_PUBLIC_API_URL}/api/super-admin/dashboard/`,
    { headers: { Authorization: `Bearer ${getAdminToken()}` } }
  );
  if (!res.ok) throw new Error("Dashboard fetch failed");
  return res.json();
}
```

Screen 2 — School Management: What to Replace

Hardcoded Value	Replace With	Notes
SCHOOLS array (9 objects)	GET /api/super-admin/schools/	Paginated, 25 per page
Filter chips state	URL query params (?status=active&board;=CBSE)	Preserve on browser back
Search input (no logic)	Debounced ?search= query param (300ms)	Searches name, tenantId, gstin

Hardcoded Value	Replace With	Notes
Seat usage bar (static %)	<code>school.students / school.seats * 100</code>	Computed from API data
"Add School" form (no POST)	<code>POST /api/super-admin/schools/provision/</code>	Returns <code>tenant_id</code>
Tenant ID field (static TNT_...)	Response from provision API — display read-only	Locked after creation
Plan badges	<code>school.plan</code> field from API	<code>trial / premium / enterprise</code>
Activity timestamp	<code>school.lastActivity</code> from API	Format with <code>date-fns</code>

Screen 3 — Billing: What to Replace

Hardcoded Value	Replace With	Notes
Invoice list (6 objects)	<code>GET /api/super-admin/billing/invoices/</code>	Paginated, sortable by date/status
Seller GSTIN (hardcoded 29...)	Global Policy: <code>gst.seller_gstin</code> from <code>/api/super-admin/policies/</code>	Never hardcode GST numbers
IGST/CGST/SGST split logic	Backend computes and returns tax breakdown	Backend knows seller state, buyer state
MRR and trend numbers	<code>GET /api/super-admin/billing/mrr/</code>	Returns current, previous, trend%
Invoice line items	<code>school.plan</code> → subscription model on backend	SAC code 998313 for SaaS
"Amount in words" text	Backend utility function or frontend lib	Use <code>num2words-india</code> package
GSTR-1 export	<code>GET /api/super-admin/billing/export/gstr1/</code>	Returns CSV/Excel

Screen 4 — Audit Log: What to Replace

Hardcoded Value	Replace With	Notes
10 hardcoded audit events	<code>GET /api/super-admin/audit/</code>	Paginated, newest first
Filter chips (no logic)	URL query params (<code>?actor=system&action;=invoice.overdue</code>)	Multi-select filters
Date range filter	<code>?from=2026-05-01&to;=2026-05-12</code>	Use date range picker component
Static timestamps	<code>event.timestamp</code> from API (ISO 8601)	Format as "X min ago" client-side
IP addresses	<code>event.ipAddress</code> from API	Mask last octet for privacy if needed
Export CSV button	<code>GET /api/super-admin/audit/export/?format=csv</code>	Backend streams the file

Screen 5 — Global Policies: What to Replace (V2)

Hardcoded Value	Replace With	Notes
14 hardcoded policy objects	<code>GET /api/super-admin/policies/</code>	Returns key, description, value, <code>isToggle</code>
Toggle state (all on)	<code>policy.enabled</code> from API	Persist on PATCH
Policy values (e.g. "10")	<code>policy.value</code> from API	Inline edit → PATCH on save
Save Changes (no-op)	<code>PATCH /api/super-admin/policies/</code> with changed keys	Audit-log every policy change
Export Config button	<code>GET /api/super-admin/policies/export/</code>	Returns JSON/YAML config file

06

Advantages of the Existing Screen Design

Why this prototype is already a strong foundation and what it saves you

Design System Is Already Solved

The prototype uses a consistent design language: dark header band, KPI card grid, panel sections, color-coded status chips, and monospace policy keys. You do not need to design anything from scratch — just convert template literals to React components and plug in real data. This saves 2–3 weeks of UI design time.

GST Logic Is Already Modelled

The billing screen already implements the IGST vs CGST+SGST split based on buyer/seller state codes. Moving this logic to the Django backend is a direct translation. The UI just needs to render the tax breakdown returned by the API.

Audit Trail Vocabulary Is Defined

The 10 audit events in the prototype cover the real action types you need to log: school.provision, plan.upgrade, auth.impersonate, invoice.overdue, api_key.rotate, school.archive. These become your AuditEvent model choices — zero guesswork.

Policy Groups Map to Django Settings

The 3 policy groups (Security, Data Isolation, Billing) match exactly the categories you want in a GlobalPolicy model. Each policy key is already in dotted-namespace format (password.min_length, gst.rate) which maps cleanly to a key-value store or Django config.

Multi-Tenancy Architecture Is Documented

The PDF brief and the School Management screen together give you the complete django-tenants model, middleware stack, provisioning API, and S3 storage backend. An intern can implement the full provisioning flow in one sprint using the code blueprints.

Filter and Search Patterns Are Defined

The School Management screen shows 5 filter dimensions (status, board, plan, region, state) plus a search box for name/tenantId/GSTIN. Converting these to URL query params and Django filter backends is a standard pattern — no design decisions needed.

07 Impact on Existing Code

What to touch, what to leave completely alone, and what to add fresh

File / Module	Action	Why
app/(dashboard)/**	LEAVE UNTOUCHED	School ERP screens are unrelated to super-admin
frontend/lib/api/ (existing)	LEAVE UNTOUCHED	School-level API clients are already scoped correctly
frontend/components/ (existing)	READ-ONLY — reuse Button, Badge, Input, Table, KPICard	Reuse existing components to match design system
frontend/styles/tokens.css	READ-ONLY — reference only, never edit	Source of truth for colors — never hardcode hex values
app/(super-admin)/ (new)	CREATE NEW — entire folder	All super-admin routes live here, zero overlap with school routes
lib/api/super-admin/ (new)	CREATE NEW — 4 API client files	schools.ts, billing.ts, audit.ts, policies.ts
backend/tenants/ (new Django app)	CREATE NEW — SchoolTenant model + views	Provision, list, update school tenants
backend/settings.py	EDIT — 4 lines only	Add django_tenants to INSTALLED_APPS, set TENANT_MODEL, swap DB engine, add middleware
backend/urls.py	EDIT — add super-admin URL prefix	Route /api/super-admin/* to new views
backend/students/models.py	LEAVE UNTOUCHED	django-tenants isolates automatically — no model changes needed
backend/middleware.py	EDIT — add TenantMainMiddleware first	Must be the FIRST middleware in the stack

Critical Rule: No School Filtering on Frontend

The most common intern mistake is adding `WHERE school_id = current_school` type filtering in frontend code or duplicating it in the backend views. **Never do this.** django-tenants handles all tenant isolation at the DB layer by switching the PostgreSQL `search_path`. All you need to do is ensure the correct subdomain is in the request — the middleware does the rest. If you add manual `school_id` filtering, you will get double-filtering bugs and break multi-tenant switching.

08

V1 vs V2 — What to Ship First

Essential screens for launch vs features that can safely wait for version 2

Not all 5 screens are equally urgent. Here is how to prioritise based on what blocks the product from going live with real schools.

Screen	Version	Justification
1. Super Admin Dashboard	V1 — SHIP FIRST	Eskoolia's own team needs visibility into all tenant health, MRR, and alerts from day one. Without this, platform operations are blind.
2. School Management (list + provision)	V1 — SHIP FIRST	You literally cannot add a new school without this screen being functional. It is the entry point for every new tenant. The provisioning form calls /api/tenants/provision/ which creates the DB schema.
3. Tenants & Billing (invoices)	V1 — SHIP FIRST	Revenue collection and GST compliance are legal requirements. You need to generate GST invoices for every school from the first billing cycle. GSTR-1 export is needed for quarterly GST filings.
4. Audit Log	V1 — SHIP FIRST	The PDF brief states: "Logs retained 7 years — required for data isolation compliance." Without this, you cannot prove to enterprise/government clients that platform-level actions are recorded. Also critical for debugging production issues.
5. Global Policies	V2 — DEFER	Platform-wide defaults can be managed in Django admin or in a config file initially. None of the schools are affected by this screen not existing yet. Build this in Sprint 3+ once the first four screens are stable and tested.

Sub-Features Within V1 Screens to Defer to V2

Screen	V1 Feature (ship)	V2 Feature (defer)
Dashboard	KPI cards + total counts + alert count	Sparkline charts, MoM trend graph, board breakdown chart
School Management	List view + provision form + status chips	Inline edit, bulk actions, advanced filter combinations
Billing	Invoice list + GST invoice view + basic MRR KPI	GSTR-1 export, dunning automation, subscription upgrade flow
Audit Log	Paginated list + actor/action filter	Date range picker, IP geolocation, export to CSV

09 Implementation Route-Map

Sprint-by-sprint plan — follow these steps in order. Each step builds on the previous.

This is written for a junior engineer or intern who is new to the codebase. Each step is self-contained. Estimated time per step is shown. Complete each step fully before moving to the next.

Sprint 0 — Setup (0.5 days)

0.1 Read tokens.css completely

Open frontend/styles/tokens.css. Memorise the color variables (--pu, --ink-1, --bd, --err). You will use these in every component. Never use hex values directly.

0.2 Read StudentList.tsx

This is the gold standard for list pages. Understand how it handles loading state (shimmer), empty state, error state, and pagination. Your school list screen follows the same pattern.

0.3 Install django-tenants

Run: `pip install django-tenants`. Follow Section 4.1-4.3 of the PDF brief exactly. Edit settings.py to add to `INSTALLED_APPS`, set `TENANT_MODEL`, swap the DB engine, and add `TenantMainMiddleware` as the FIRST middleware.

0.4 Create the tenants Django app

`python manage.py startapp tenants`. Create `SchoolTenant` model from the blueprint in Section 3.1 of the PDF brief. Run `python manage.py migrate_schemas --shared`.

Sprint 1 — Backend API (2–3 days)

1.1 Create IsSuperAdmin permission class

In `tenants/permissions.py`: `class IsSuperAdmin(BasePermission): return request.user.is_staff and request.user.role == "super_admin"`. Use this on EVERY super-admin view.

1.2 Build /api/super-admin/schools/ (GET + POST)

GET returns paginated `SchoolTenant` list with filters (status, board, plan, search). POST calls `ProvisionSchoolView` from the PDF brief blueprint (Section 4.4). Returns `tenant_id` on success.

1.3 Build /api/super-admin/dashboard/ (GET)

Aggregates School count, student totals, MRR sum, alert count, and board breakdown from `SchoolTenant.objects.all()`. Also queries last 5 audit events for recent activity.

1.4 Build /api/super-admin/billing/invoices/ (GET + POST)

GET returns Invoice list. POST creates a new invoice, auto-computing IGST vs CGST+SGST based on buyer.state vs seller state (from Global Policy `gst.seller_state`).

1.5 Build /api/super-admin/audit/ (GET)

Returns `AuditEvent` list filtered by actor, action, tenant, and date range. Paginated, newest first. The `AuditEvent` model: `id, timestamp, actor, action, detail, ipAddress, tenantId`.

1.6 Wire up URL patterns

In `backend/urls.py` add: `path("api/super-admin/", include("tenants.urls_super_admin"))`. Create `tenants/urls_super_admin.py` with all the routes.

Sprint 2 — Frontend: API Client Layer (1 day)

2.1 Create lib/api/super-admin/schools.ts

getSchools(params), provisionSchool(data), updateSchool(id, data). Follow the exact fetch pattern from Section 6 of the PDF brief. Define TypeScript interfaces for every response shape (no any).

2.2 Create lib/api/super-admin/dashboard.ts

getDashboard(). Returns DashboardData interface (see Section 05 of this guide).

2.3 Create lib/api/super-admin/billing.ts

getInvoices(params), createInvoice(data), getMrr(). Define Invoice, MrrData interfaces.

2.4 Create lib/api/super-admin/audit.ts

getAuditEvents(params), exportAuditCsv(params). Define AuditEvent interface matching the Django model fields.

2.5 Create (super-admin)/layout.tsx

Checks JWT role === "super_admin". If not, redirect("/login"). Renders the sidebar nav (Dashboard / Schools / Billing / Audit / Policies links). Reuse the existing Sidebar component pattern from the school layout.

Sprint 3 — Frontend: Screens (3–4 days)

3.1 Build Dashboard page (page.tsx)

useEffect → getDashboard() → loading state (shimmer 4 KPI cards) → render real data. KPI cards: total schools, students, MRR, alerts. Board breakdown: simple horizontal bar (use Recharts BarChart). Recent activity: map over last 5 audit events from the API.

3.2 Build School Management list page

useEffect → getSchools() → shimmer skeleton rows → render school cards. Wire up filter chips (status/board/plan) to update URL query params. Wire up search input with 300ms debounce. Clicking a card opens a detail panel (useState for selectedSchool).

3.3 Build School Provisioning form (modal or panel)

Form with: School Name, Subdomain, State, Board, Plan, DB Shard, Storage Region, SSO Method, API Access toggle, Backup Retention. On submit → provisionSchool(data) → show returned tenant_id as read-only. Disable subdomain field after successful creation.

3.4 Build Billing page

MRR KPI row + invoice table. Each invoice row: number, school name, date, amount, GST status chip. Clicking a row opens the full GST invoice panel with line items, tax split, and amount in words. Export GSTR-1 button calls exportGstr1() and triggers file download.

3.5 Build Audit Log page

Filter bar: actor select, action multi-select, date range pickers. Paginated event list with color-coded icons. Each event: timestamp, actor badge, action code (monospace), detail text, IP. Export CSV button calls exportAuditCsv() and triggers download.

Sprint 4 — V2: Global Policies (1–2 days)

4.1 Create GlobalPolicy Django model

Fields: key (CharField, unique), description, value (TextField), is_toggle (BooleanField), is_overridable (BooleanField), group (CharField with choices). Seed with the 14 policies from the prototype.

4.2 Build /api/super-admin/policies/ (GET + PATCH)

GET returns all policies grouped. PATCH accepts { key: new_value } object, validates, saves, and logs each change to the audit trail.

4.3 Build Policies frontend page

Fetch all policies → group by policy.group → render as 3 collapsible panels. Each policy row: monospace key, description, inline value editor, toggle. Batch save on "Save Changes" button → PATCH API → invalidate cache.

10

Audit & Validation Checklist

How to verify every screen is production-ready before pushing to the Testing branch

UI/UX Audit Rules (Every Screen)

Check	How to Verify	Pass Condition
Loading state exists	Slow API in DevTools (throttle to Slow 3G)	Shimmer skeleton visible, no blank flash
Error state exists	Block API in DevTools → trigger network error	Error message visible with retry button
Empty state exists	Mock API returning empty array	Illustrated empty state, not a blank table
No hardcoded data	Search codebase for school names, GSTIN values, MRR numbers	Zero hits — all data from API
No any TypeScript types	Run: npx tsc --noEmit	Zero type errors
Design tokens used	Search for hardcoded hex values (#6D4AFF etc.)	Zero hits — only token variables
RBAC enforced	Log in as a school Principal, try /super-admin/	Redirected to login — access denied
Accessibility labels	Run axe DevTools browser extension	Zero critical violations
Mobile responsive	DevTools → toggle device toolbar (375px width)	Cards stack, no horizontal scroll

Backend Validation Checklist

Check	How to Verify
IsSuperAdmin blocks school users	Call super-admin API with a school-level JWT → expect 403 Forbidden
Tenant ID is immutable after creation	Try PATCH /api/super-admin/schools/{id}/ with tenant_id field → field should be ignored
Audit log entry created on every mutation	Provision a school → check /api/super-admin/audit/ for school.provision event
GST tax split is correct (IGST vs CGST+SGST)	Create invoice for TG school (state 36) from KA seller (state 29) → should show IGST 18%
Pagination works on all list endpoints	Seed 30 schools → GET /api/super-admin/schools/?page=2 → returns correct 5 records
django-tenants isolation holds	From school A's session, call Student.objects.all() → confirm only school A students returned
Provisioning is atomic	Simulate failure mid-provision → confirm no half-created schema exists

Final Pre-Launch Checklist

Item	Status Check
All API endpoints return correct TypeScript-matching shapes	Run frontend against real backend — no type mismatches
DB migrations run on a clean schema	python manage.py migrate_schemas --shared on fresh DB
Super-admin login works end-to-end	Login → JWT issued → dashboard loads with real data
School provisioning creates a real schema	Provision a test school → check PostgreSQL: \dn shows new schema
Invoice generation produces correct GST totals	Compare PDF invoice totals against manual calculation

Item	Status Check
Audit log shows all test actions	Review audit log — every test action should appear with correct actor and timestamp
No console errors in production build	npm run build → npm start → open all 4 V1 screens — zero console errors
Performance: list loads under 1 second	Seed 50 schools → measure /api/super-admin/schools/ response time < 500ms

You are ready to ship when all checklists above are green. Push to the Testing branch only — never directly to main. Tag the release as v0.1.0-super-admin and notify the team via WhatsApp/Slack with a link to the Vercel preview URL.